

Материалы для подготовки к зачёту по ОС 2026

Содержание

1	Практики	2
1.1	Базовые команды терминала Linux	2
1.2	Синонимы для директорий	2
1.3	Мануалы	3
1.4	Работа с правами файлов	3
2	Программы с практических занятий (за работоспособность не отвечаю)	5
2.1	Лесенка из звёздочек на bash	5
2.2	Hello World	5
2.3	Hello World с количеством повторений, заданным в аргументах командной строки	5
2.4	Вывод аргументов командной строки	5
2.5	Лесенка из звёздочек и точек	6
2.6	Зеркальная лесенка из звёздочек и точек	6
2.7	Перевод числа из двоичной в десятичную систему счисления	6
2.8	Сложение двоичных чисел	7
2.9	Умножение двоичных чисел	8
2.10	Получение текущей даты	10
2.11	Шифр Цезаря (probably doesn't work)	11
2.12	Непрерывная запись в файл	11
2.13	Непрерывная запись в файл с использованием дочернего процесса	11
2.14	Непрерывная запись в файл с помощью демона	12
2.15	Самодельный аналог sr	13
2.16	Простой обработчик сигналов	13
2.17	Продвинутый обработчик сигналов	13
2.18	Самодельный аналог kill	14
2.19	Информация о PID, PPID, SID, GRPID процесса	14
2.20	Программа, автоматически меняющая свой PID и название	15
3	Лекции	16
3.1	Введение	16
3.2	История Unix	16
3.3	Компьютерные сети	16
3.4	Уровни операционной системы	17
3.5	Системные вызовы	17

1 Практики

1.1 Базовые команды терминала Linux

- *reset* — сброс настроек терминала (например, если сломалась кодировка);
- *clear* или *Ctrl+L* — очистка экрана терминала;
- *strings <бинарный файл>* — вывод всех текстовых строк, содержащихся в бинарном файле;
- *set* — список всех системных переменных;
- *ln myfile myfile.hard* — создание жёсткой ссылки (только в пределах файловой системы);
- *ln --symbolic (-s) myfile myfile.sym* — создание символической ссылки (может вести на несуществующий файл, аналогично ярлыкам);
- *file <файл>* — информация о файле (empty/ASCII string/binary file и т.д.);
- *which <команда>* (например, *which bash*) — информация о расположении бинарного файла команды;
- *mkdir -p dir1/dir2/dir3* — создать иерархическую структуру директорий, все несуществующие директории на пути также будут созданы;
- *rmdir dir1* — удалить пустую директорию;
- *rm -r dir1* — удалить директорию вместе со всем её содержимым;
- *ls > file.txt* — перенаправление вывода в файл (если он уже существовал, то он будет перезаписан);
- *ls >> file.txt* — перенаправление вывода в файл (если он не существовал, то он будет создан, иначе добавление производится в конец существующего файла);
- *history* — история команд (можно выполнять навигацию с помощью ↑ и ↓);
- *kill -n <signal> <pid>* — послать сигнал с номером *signal* процессу *pid*;
- *ps -ef* — диспетчер задач.

1.2 Синонимы для директорий

- *~* — домашняя директория;
- *~ <username>* — домашняя директория пользователя *username* (/home/username);
- *.* — текущая директория;
- *..* — родительская директория.

1.3 Мануалы



Самое важное — это ЧИТАТЬ МАНЫ.

- `man <команда>` — ман по команде;
- `man -k <слово>` или *apropos (appropriate position) <слово>* — все страницы манов, где встретилось слово;
- `<команда> --help` или `<команда> -h` — для простых смертных, краткая справка по команде.

1.4 Работа с правами файлов

`ls -l` выводит подробную информацию обо всех файлах вместе с их флагами.

Пример флага: `-rwx-wx-w-`

Подробный разбор флага:

1. Первый символ отвечает за то, является ли файл специальным (для директории там будет стоять *d*, для символической ссылки — *l*, для обычного файла — прочерк `-`).
 2. Следующие три символа (с 2 по 4) отвечают за права владельца.
 3. Следующие три символа (с 5 по 7) отвечают за права группы.
 4. Последние три символа (с 8 по 10) отвечают за права остальных пользователей.
- *r* (Read) — право записи;
 - *w* (Write) — право чтения;
 - *x* (eXecute) — право исполнения;
 - `-` (прочерк) — такого права нет.

chown <user> <file> — передать права владения файлом пользователю user.

chmod u=rwx, g-x, o+rw <file> — дать владельцу все права, у пользователей группы забрать права на исполнение, а остальным пользователям добавить возможность читать файл и писать в него.

Флаг можно закодировать как двоично-десятичное число. Например, флаг rwx--x-wx будет закодирован как $111_2 001_2 011_2 = 713_{(10)}$.

chmod 777 <file> — дать на файл все права всем пользователям.

2 Программы с практических занятий (за работоспособность не отвечаю)

2.1 Лесенка из звёздочек на bash

```
#!/bin/bash

str="*"
for i in {1..10}; do
    echo "$str"
    str="$str*"
done
```

2.2 Hello World

```
#include <stdio.h>

int main() {
    int i, n;
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Hello, world!\n");
    }
    return 0;
}
```

2.3 Hello World с количеством повторений, заданным в аргументах командной строки

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int i;
    if (argc < 2) return 0;
    int n = atoi(argv[1]);
    for (i = 0; i < n; i++) {
        printf("Hello, world!\n");
    }
    return 0;
}
```

2.4 Вывод аргументов командной строки

```
#include <stdio.h>

int main(int argc, char* argv[]) {
```

```

    int i;
    for (i = 0; i < argc; i++) {
        printf("argv[%d] = %s \n", i, argv[i]);
    }
    return 0;
}

```

2.5 Лесенка из звёздочек и точек

```

#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int i, j;
    int n = atoi(argv[1]);
    for (i = 0; i < n; i++) {
        for (j = 0; j <= i; j++) {
            printf((j % 2 == 0 ? "*" : "."));
        }
        printf("\n");
    }
    return 0;
}

```

2.6 Зеркальная лесенка из звёздочек и точек

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int i, j, n;
    scanf("%d", &n);
    for (i = 0; i < 2 * n + 1; i++) {
        for (j = 0; j <= (i < n ? i : 2 * n - i); j++) {
            printf((j % 2 == 0 ? "*" : "."));
        }
        printf("\n");
    }
    return 0;
}

```

2.7 Перевод числа из двоичной в десятичную систему счисления

```

#include <stdio.h>
#include <stdlib.h>

int main() {

```

```

int bin, pw, ans, digit;
scanf("%d", &bin);
pw = 1;
ans = 0;
while (bin != 0) {
    digit = bin % 10;
    ans += (digit * pw);
    bin /= 10;
    pw <<= 1;
}
printf("%d\n", ans);
return 0;
}

```

2.8 Сложение двоичных чисел

```

#include <stdio.h>
#include <stdlib.h>

int to_dec(int bin) {
    int pw, ans, digit;

    pw = 1;
    ans = 0;
    while (bin != 0) {
        digit = bin % 10;
        ans += (digit * pw);
        bin /= 10;
        pw <<= 1;
    }
    return ans;
}

int add(int bin1, int bin2) {
    int d1, d2, d, carry, pw, ans;

    pw = 1;
    carry = 0;
    ans = 0;
    while (bin1 != 0 && bin2 != 0) {
        d1 = bin1 % 10;
        d2 = bin2 % 10;
        d = (d1 + d2 + carry) % 2;
        carry = (d1 + d2 + carry) / 2;
        ans += (d * pw);
        bin1 /= 10;
        bin2 /= 10;
    }
}

```

```

        pw *= 10;
    }
    if (bin1 == 0) bin1 = bin2;
    while (bin1 != 0) {
        d1 = bin1 % 10;
        d = (d1 + carry) % 2;
        carry = (d1 + carry) / 2;
        ans += (d * pw);
        bin1 /= 10;
        pw *= 10;
    }
    while (carry) {
        ans += (carry % 2) * pw;
        carry /= 2;
        pw *= 2;
    }
    return ans;
}

int main() {
    int num1, num2, ans, nd1, nd2, ansd;
    scanf("%d", &num1);
    scanf("%d", &num2);
    ans = add(num1, num2);
    printf("Sum: %d\n", ans);
    nd1 = to_dec(num1);
    nd2 = to_dec(num2);
    ansd = to_dec(ans);
    printf("%d\n", nd1);
    printf("%d\n", nd2);
    printf("My sum: %d\n", ansd);
    printf("Sum: %d\n", nd1 + nd2);
    return 0;
}

```

2.9 Умножение двоичных чисел

```

#include <stdio.h>
#include <stdlib.h>

int to_dec(int bin) {
    int pw, ans, digit;

    pw = 1;
    ans = 0;
    while (bin != 0) {
        digit = bin % 10;

```



```

        ans += (digit * pw);
        bin /= 10;
        pw <<= 1;
    }
    return ans;
}

int add(int bin1, int bin2) {
    int d1, d2, d, carry, pw, ans;

    pw = 1;
    carry = 0;
    ans = 0;
    while (bin1 != 0 && bin2 != 0) {
        d1 = bin1 % 10;
        d2 = bin2 % 10;
        d = (d1 + d2 + carry) % 2;
        carry = (d1 + d2 + carry) / 2;
        ans += (d * pw);
        bin1 /= 10;
        bin2 /= 10;
        pw *= 10;
    }
    if (bin1 == 0) bin1 = bin2;
    while (bin1 != 0) {
        d1 = bin1 % 10;
        d = (d1 + carry) % 2;
        carry = (d1 + carry) / 2;
        ans += (d * pw);
        bin1 /= 10;
        pw *= 10;
    }
    while (carry) {
        ans += (carry % 2) * pw;
        carry /= 2;
        pw *= 2;
    }
    return ans;
}

int mul(int bin1, int bin2) {
    int d1, d2, cur, ans, tmp, pw, pw1;

    ans = 0;
    pw1 = 1;
    while (bin1 != 0) {
        d1 = bin1 % 10;

```

```

        tmp = bin2;
        cur = 0;
        pw = 1;
        while (tmp != 0) {
            d2 = tmp % 10;
            cur += (d1 * d2 * pw);
            tmp /= 10;
            pw *= 10;
        }
        bin1 /= 10;
        ans = add(ans, cur * pw1);
        pw1 *= 10;
    }
    return ans;
}

int main() {
    int num1, num2, ans, nd1, nd2, ansd;
    scanf("%d", &num1);
    scanf("%d", &num2);
    ans = mul(num1, num2);
    printf("Product: %d\n", ans);
    nd1 = to_dec(num1);
    nd2 = to_dec(num2);
    ansd = to_dec(ans);
    printf("%d\n", nd1);
    printf("%d\n", nd2);
    printf("My product: %d\n", ansd);
    printf("Product: %d\n", nd1 * nd2);
    return 0;
}

```

2.10 Получение текущей даты

```

#include <stdio.h>
#include <time.h>

int main() {
    time_t t = time(NULL);
    struct tm *result;
    result = localtime(&t);
    printf("%d:%d:%d", result->tm_hour, result->tm_min, result->tm_sec);
    return 0;
}

```

2.11 Шифр Цезаря (probably doesn't work)

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    char buffer[100];
    int i, shift;
    freopen(argv[1], "r", stdin);
    shift = atoi(argv[2]);
    while (fread(buffer, sizeof(*buffer), 100, stdin)) {
        for (i = 0; i < 100; i++) {
            printf("%c", (buffer[i] + shift) % 26);
        }
    }
}
```

2.12 Непрерывная запись в файл

```
#include <stdio.h>
#include <stdbool.h>
#include <unistd.h>
#include <signal.h>

void handler(int n) {
    printf("ok\n");
}

int main() {
    int i = 0;
    signal(SIGHUP, handler);
    while (true) {
        freopen("myfile.txt", "a", stdout);
        printf("%d\n", i);
        i++;
        i %= 10;
        sleep(1);
    }
    return 0;
}
```

2.13 Непрерывная запись в файл с использованием дочернего процесса

```
#include <stdio.h>
#include <stdbool.h>
#include <unistd.h>
#include <signal.h>
```

```

#include <stdlib.h>

void handler(int n) {
    printf("ok\n");
}

int main(int argc, char* argv[]) {
    int i = 0;
    signal(SIGHUP, handler);
    freopen("myfile.txt", "a", stdout);
    pid_t fr = fork();
    system(argv[0]);
    if (fr != 0) {
        while (true) {
            printf("%d\n", i);
            i++;
            i %= 10;
            sleep(1);
        }
    }
    else exit();
    return 0;
}

```

2.14 Непрерывная запись в файл с помощью демона

```

#include <stdio.h>
#include <stdbool.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int i = 0;
    FILE *in = fopen("myfile.txt", "w");
    daemon(0, 0);
    while (true) {
        sleep(1);
        fprintf(in, "%d\n", i);
        fflush(in);
        i = (i + 1) % 10;
    }
    fclose(in);
    return 0;
}

```

2.15 Самодельный аналог `cp`

```
#include <stdio.h>
#include <stdbool.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char* argv[]) {
    int count = 100;
    int to_read;
    char buf[count];
    int in, out;
    if (argc < 3) {
        printf("Too few arguments\n");
        return 0;
    }
    in = open(argv[1], O_RDONLY);
    out = open(argv[2], O_CREAT | O_WRONLY);
    while ((to_read = read(in, buf, count)) > 0) {
        write(out, buf, to_read);
    }
    close(in);
    close(out);
    return 0;
}
```

2.16 Простой обработчик сигналов

```
#include <stdio.h>
#include <signal.h>
#include <stdbool.h>

void handler(int n) {
    printf("Signal N = %d received\n", n);
}

int main() {
    signal(SIGINT, handler);
    signal(SIGHUP, handler);
    signal(SIGQUIT, handler);
    while (true) {
    }
    return 0;
}
```

2.17 Продвинутый обработчик сигналов

```
#include <stdio.h>
```

```

#include <signal.h>
#include <stdbool.h>
#include <time.h>

time_t t;
bool fl = 1;

void handler_1(int n) {
    printf("Hello, world!");
}

void handler_2(int n) {
    struct tm *result;
    result = localtime(&t);
    printf("%d:%d:%d", result->tm_hour, result->tm_min, result->tm_sec);
}

void handler_3(int n) {
    fl = 0;
}

int main() {
    freopen("myfile.txt", "w", stdout);
    t = time(NULL);
    signal(SIGHUP, handler_1);
    signal(SIGINT, handler_2);
    signal(SIGQUIT, handler_3);
    while (fl) {
    }
    return 0;
}

```

2.18 Самодельный аналог kill

```

#include <stdio.h>
#include <signal.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int sig = atoi(argv[1]);
    int pid = atoi(argv[2]);
    kill(pid, sig);
}

```

2.19 Информация о PID, PPID, SID, GRPID процесса

```

#include <stdio.h>

```

```

#include <unistd.h>
#include <stdbool.h>

int main() {
    pid_t pid = getpid();
    pid_t ppid = getppid();
    pid_t sid = getsid(0);
    pid_t ssid = getsid(pid);
    pid_t grp = getpgrp();
    pid_t pgrp = getpgid(pid);
    printf("%d %d\n", pid, ppid);
    printf("%d %d\n", sid, ssid);
    printf("%d %d\n", grp, pgrp);
    while (true) {
    }
}

```

2.20 Программа, автоматически меняющая свой PID и название

```

#include <stdio.h>
#include <stdbool.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    pid_t fr;
    freopen("test.txt", "a", stdout);
    argv[0][6] = '0';
    while (true) {
        printf("%d\n", getpid());
        sleep(1);
        fr = fork();
        argv[0][6] = (char)(argv[0][6] + 1);
        if (argv[0][6] > '9') argv[0][6] = '0';
        if (fr == 0) execl("a.out", argv[0], NULL);
        else exit(0);
    }
    return 0;
}

```

3 Лекции

3.1 Введение

Компьютеры делятся на цифровые вычислительные машины (ЦВМ) и аналоговые вычислительные машины (АВМ). Планировщик распределяет время между процессами с помощью карусели/приоритетов, т.е. применяются квазипараллельные вычисления (в каждый момент времени процессор занят одной задачей, но эти задачи постоянно меняются).

Операционные системы делятся на универсальные ОС и ОС реального времени. Существуют однопользовательские и многопользовательские ОС.

Сервис — подпроцесс, работающий самостоятельно и изолированный от других.

3.2 История Unix

Multics. Первая ОС, написанная на языке высокого уровня (C) — Unix. А. Танненбаум — Minix. Линус Торвалдс — Linux (монолитное ядро, в результате спора с Танненбаумом Linux остался обозначением для ядра, а не для полноценной ОС). FreeBSD. QNX (Quick Unix) — ОС реального времени с микроядерной архитектурой.

Linux — это только ядро, мы же пользуемся *дистрибутивами* Linux. Дистрибутив включает в себя ядро, пакетный менеджер и набор прикладных программ.

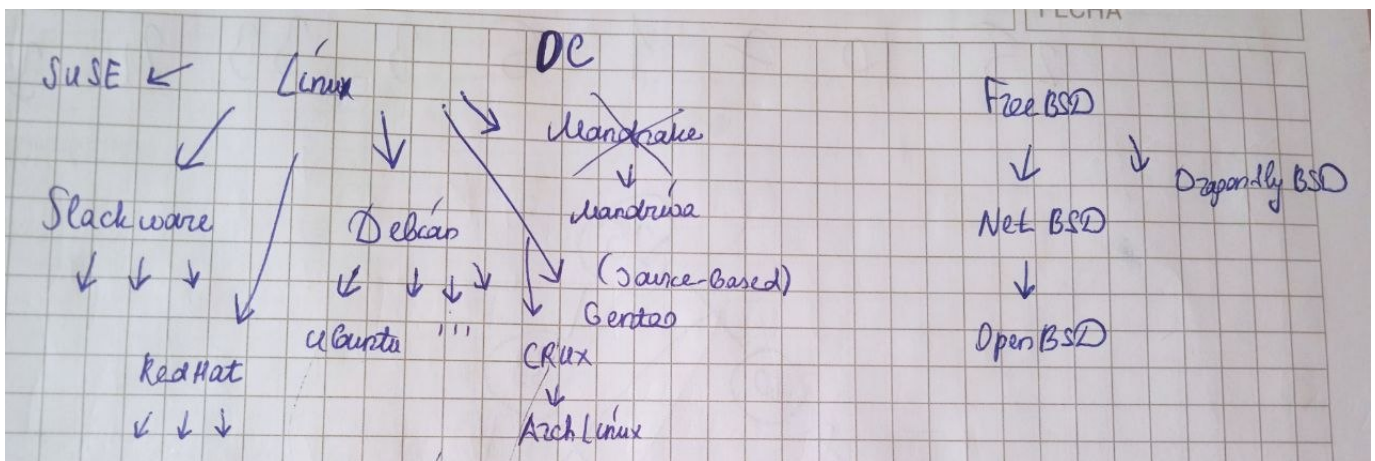


Рис. 1: Схема дистрибутивов Linux и FreeBSD

LFS — Linux From Scratch, возможность создать собственный дистрибутив.

LSB — Linux Standard Base.

SUS — Single Unix Specification.

3.3 Компьютерные сети

Коммутация каналов/коммутация пакетов.

Протокол — соглашение по передаче-обработке данных.

RFC — requests for comments.

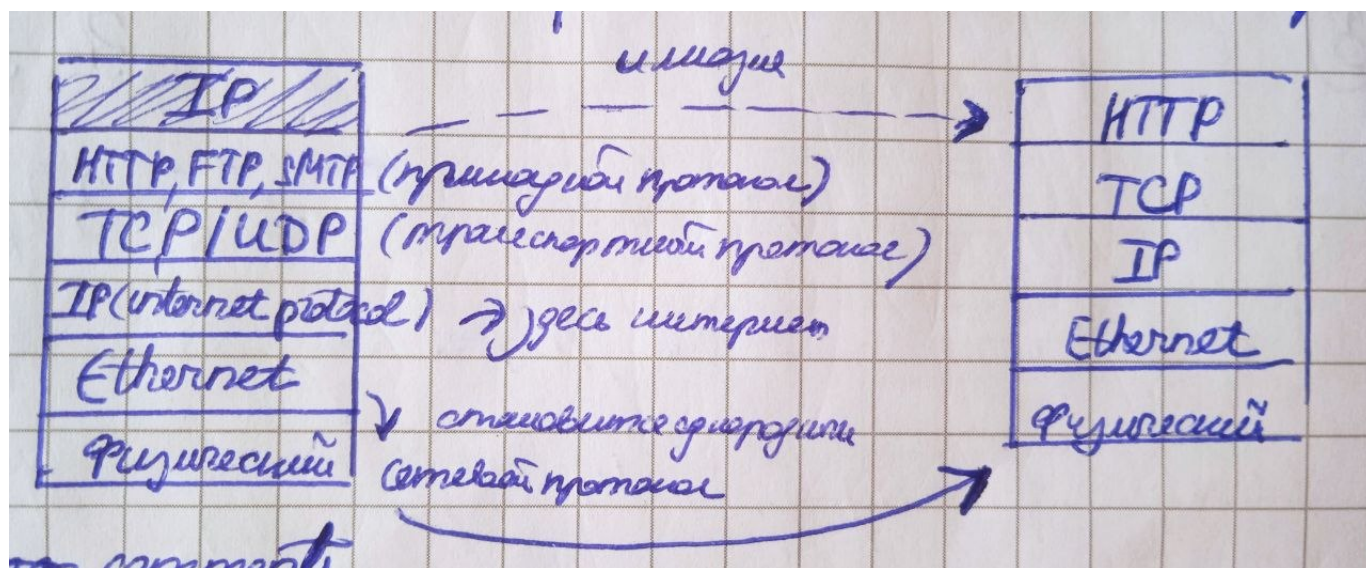


Рис. 2: Стек протоколов

3.4 Уровни операционной системы

1. Kernel — ядро, внутренний уровень;
2. Shell — оболочка, внешний уровень.

X Window System — графическая оболочка.

XLog — окно входа X Server.

3.5 Системные вызовы

Аналог команды `ср` с помощью высокоуровневых функций работы с файлами:

```
FILE* fv = fopen("from.txt", "r");
FILE* tv = fopen("to.txt", "w");
while ((c = getc(fv)) != EOF) putc(c, tv);
fclose(fv);
fclose(tv);
```

Системные вызовы для работы с файлами:

- Создание файла:

```
creat("file.txt", 0642);
```

- Сброс прав по умолчанию:

```
umask(0);
```

- Создание файла и запись в него:

```
char buf[10];
umask(0);
int fd = creat("file.txt", 0642);
write(fd, buf, sizeof(buf));
close(fd);
```

- Открытие файла (старый способ):

```
int mode = 1; // 0 - r, 1 - w, 2 - rw
open("file.txt", mode);
```

- Открытие файла (новый способ): Применяются флаги, их можно комбинировать через побитовое или (|).

1. O_CREAT — создать;
2. O_RDONLY — открыть только для чтения;
3. O_WRONLY — открыть только на запись;
4. O_RDWR — открыть на чтение и запись;
5. O_APPEND — запись в конец файла;
6. O_NDELAY — не блокировать при открытии;
7. O_TRUNC — сократить размер файла до нуля.

```
int fd = open("from.txt", O_RDONLY);
int to = open("to.txt", O_WRONLY | O_CREAT);
char buf[10];
while ((n = read(fd, buf, sizeof(buf))) > 0) {
    write(to, buf, n);
}
```

- Перенос указателя: Сдвинуть указатель файлового дескриптора *fd* на 3 байта вправо с начала.

```
lseek(fd, 3, 0);
```

Стандартные потоки: 0 — stdin, 1 — stdout, 2 — stderr.